

Code Explanation

DP Address Change YUYU Processor Test Suite

This test suite is designed to validate the functionality of the `DPAddressChangeYUYU` processor, which is responsible for processing address change data. The tests cover various scenarios, including checking source records, processing YUYU creation, writing output, and handling empty source data.

Overview of the Code

The code is a comprehensive test suite for the `DPAddressChangeYUYU` processor, utilizing PySpark and Pytest. It includes fixtures for creating a `SparkSession`, temporary directory, test configuration, and sample data. The tests validate the processor's methods, ensuring correct functionality and output.

Step 1: Creating a SparkSession Fixture

The first step involves creating a `SparkSession` fixture, which is a crucial component for testing PySpark applications. This fixture is defined with a module scope, meaning it will be created once for the entire test module.

```
@pytest.fixture(scope="module")
def spark():
    """Create a SparkSession for testing."""
    return SparkSession.builder
        .appName("DPAddressChangeYUYU-Test")
        .master("local[2]")
        .getOrCreate()
```

Step 2: Creating a Temporary Directory Fixture

The next step is to create a temporary directory fixture using the `tempfile` module. This directory will be used for storing test outputs.

```
@pytest.fixture(scope="module")
def temp_dir():
    """Create a temporary directory for test outputs."""
    with tempfile.TemporaryDirectory() as tmp_dir:
        yield tmp_dir
```

Step 3: Creating Test Configuration

The test configuration is created as a fixture, which includes the target directory and batch size. This configuration is used throughout the tests.

```
@pytest.fixture(scope="module")
def config(temp_dir):
    """Create test configuration."""
    return {
        "target_dir": temp_dir,
        "batch_size": 1000
    }
```

Step 4: Creating Sample Test Data

The sample test data is created using PySpark DataFrames and includes the necessary schema and data for testing. The data is registered as temporary views for use in the tests.

```
@pytest.fixture(scope="module")
def sample_data(spark):
    """Create sample test data."""
    # Create schema for test data
    schema = StructType([
        StructField("T_TX_BASIC_POLICY_ID", StringType(), True),
        # ...
    ])

    # Create test data
    data = [
        (
            "POL123456", "REQ123456", "2023-01-15 10:30:00",
            "123-4567", "1", "2", "3",
            "1", "2", "3", "08012345678"
        ),
        # ...
    ]

    df = spark.createDataFrame(data, schema)
    df.createOrReplaceTempView("STG_E2E_AC_TXDBH_DATA")

    # Create lookup tables
    kana_schema = StructType([
        StructField("POL_NO", StringType(), True),
        StructField("POWN_LNM", StringType(), True),
        StructField("POWN_FNM", StringType(), True)
    ])

    # ...
```

Step 5: Testing the check_source_records Method

The `test\_check\_source\_records` function tests the `check\_source\_records` method of the `DPAAddressChangeProcessor` class. It verifies that the correct count of source records is returned and that the count file is created.

```
def test_check_source_records(spark, sample_data, config):
    """Test the check_source_records method."""
    processor = DPAAddressChangeProcessor(spark, config)
    count = processor.check_source_records()

    assert count == 2, "Should find 2 records in the source data"

    # Check that the count file was created
    count_file_path = os.path.join(config["target_dir"], "FF_Source_
Count.out")
    assert os.path.exists(count_file_path), "Count file should be cr
eated"
```

Step 6: Testing the process_yuyu_creation Method

The `test\_process\_yuyu\_creation` function tests the `process\_yuyu\_creation` method, verifying that the output DataFrame has the correct number of rows and contains all required columns.

```
def test_process_yuyu_creation(spark, sample_data, config):
    """Test the process_yuyu_creation method."""
    processor = DPAddressChangeProcessor(spark, config)
    yuyu_df, staging_df = processor.process_yuyu_creation()

    # Check that the output DataFrame has the correct number of rows
    assert yuyu_df.count() == 2, "Output should have 2 rows"

    # Check that all required columns are present
    required_columns = [
        "SEQUENCE_NUMBER", "ORIGIN_REQUEST_ID", "POLICY_ID", "RECEPTION_DATE",
        # ...
    ]

    for col_name in required_columns:
        assert col_name in yuyu_df.columns, f"Column {col_name} should be present in output"
```

Step 7: Testing the write_yuyu_output Method

The `test\_write\_yuyu\_output` function tests the `write\_yuyu\_output` method, verifying that the output file is created.

```
def test_write_yuyu_output(spark, sample_data, config):
    """Test the write_yuyu_output method."""
    processor = DPAddressChangeProcessor(spark, config)
    yuyu_df, _ = processor.process_yuyu_creation()

    # Write the output
    processor.write_yuyu_output(yuyu_df)

    # Check that the output file was created
    output_path = os.path.join(config["target_dir"])

    # In a real test, we would check for the specific file
    # Here we just check that some files were created
    assert os.listdir(output_path), "Output directory should not be empty"
```

Step 8: Testing Empty Source Data Handling

The `test\_empty\_source\_handling` function tests the handling of empty source data, verifying that the correct count is returned and that a trigger file is created.

```
def test_empty_source_handling(spark, config):
    """Test handling of empty source data."""
    # Create empty source data
    empty_schema = StructType([
        StructField("TX BASIC POLICY ID", StringType(), True),
```

```

        StructField("T_TX_REQUEST_ORIGIN_REQUEST_ID", StringType(),
True)
    ])

    empty_df = spark.createDataFrame([], empty_schema)
    empty_df.createOrReplaceTempView("STG_E2E_AC_TXDBH_DATA")

    processor = DPAddressChangeProcessor(spark, config)
    count = processor.check_source_records()

    assert count == 0, "Should find 0 records in the empty source da
ta"

    # Test trigger file creation
    processor.create_empty_trigger_file()
    trigger_path = os.path.join(config["target_dir"], "trigger_file"
)
    assert os.path.exists(trigger_path), "Trigger file directory sho
uld be created"

```